ARQUITECTURA DE COMPUTADORES

GRADO EN INGENIERÍA INFORMÁTICA – INGENIERÍA DEL SOFTWARE

ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA INFORMÁTICA

DEPARTAMENTO DE ARQUITECTURA Y TECNOLOGÍA DE COMPUTADORES

UNIVERSIDAD DE SEVILLA

http://www.atc.us.es

Autor: José Luis Guisado Lizar.
Mayo de 2012
Revisión usando Python: Miguel Angel Rodríguez Jódar
Febrero de 2024

INTRODUCCIÓN A MPI

- MPI (Message-Passing Interface) es una biblioteca de funciones que implementan la **ejecución distribuida** de un programa mediante **paso de mensajes**
  - Pueden ser llamadas desde un programa en C, FORTRAN, Python, MATLAB, entre otros.

- MPI es realmente la especificación de una interfaz (API) de dicha biblioteca de funciones
- Estándar definido y mantenido por el “MPI Forum”, consorcio integrado por representantes de muchas organizaciones
- Existen múltiples implementaciones reales del estándar MPI:
  - MPICH, LAM/MPI, OpenMPI...

INTRODUCCIÓN A MPI

- MPI: biblioteca de funciones que implementan la **ejecución distribuida** de un programa mediante **paso de mensajes**

- MODELO DE PROGRAMACIÓN DE MEMORIA COMPARTIDA:
  - Múltiples hilos ejecutándose en paralelo
  - Comparten el espacio de direcciones de memoria
  - Comunicación mediante variables compartidas
  - Ejemplo: OpenMP
- MODELO DE PROGRAMACIÓN DE MEMORIA DISTRIBUIDA:
  - Múltiples procesos ejecutándose en paralelo
  - Cada proceso tiene un espacio de direcciones de memoria propio
  - Comunicación mediante paso de mensajes
  - Ejemplos: MPI, PVM

En Python, MPI opera a través de llamadas a funciones, muchas de ellas métodos de un comunicador. Otras son métodos estáticos.

Para indicar al intérprete que queremos usar MPI añadimos esto al comienzo del script:

```
from mpi4py import MPI
```

Funciones MPI:

Métodos estáticos: `MPI.nombre_método(args)`

Métodos de un comunicador: `comm.nombre_método (args)`

Pueden lanzar una excepción (no hay códigos de error):

```
try:
    ... # llamadas a funciones MPI
    ...
except Exception as e:
    print (f"Excepción: {e}")
```

- **COMUNICADORES**:
  - Un comunicador define un grupo de procesos que están involucrados en una operación MPI (normalmente algún tipo de comunicación)
  - Todas las funciones de comunicación de MPI son métodos de un objeto comunicador.
  - El comunicador predefinido **MPI.COMM_WORLD**:
    - Incluye todos los procesos que se ejecutan desde mpirun
    - Es el utilizado al programar en MPI a nivel básico

Primero establecemos un comunicador, así:

```
comm = MPI.COMM_WORLD
```

Y ya podemos usar métodos de ese comunicador

- **comm.Get_size** – ¿Cuántos procesos hay?
- **comm.Get_rank** – Mi número de proceso
- **comm.send** – Enviar un mensaje
- **comm.recv** – Recibir un mensaje

`rank = comm.Get_rank()`

- Devuelve el rango del proceso actual en un comunicador.
- Cada comunicador contiene un grupo de procesos, cada uno de los cuales tiene un identificador único (rango), que es un número entero que comienza en 0: (0,1,2,3,...).

`size = comm.Get_size()`

- Devuelve el número de procesos que participan en el comunicador comm. Si éste es el comunicador global predefinido MPI.COMM_WORLD, indica el número total de procesos involucrados en el programa

PRIMER EJEMPLO BÁSICO : hola_mundo.py


```python
from mpi4py import MPI

# Establezco el comunicador global
comm = MPI.COMM_WORLD
# Qué numero de proceso soy?
rank = comm.Get_rank()
# Cuántos procesos hay en mi comunicador?
size = comm.Get_size()

print (f"Hola, mundo! Soy el proceso {rank} de {size}")
```

PRIMER EJEMPLO BÁSICO : programa hola_mundo.py

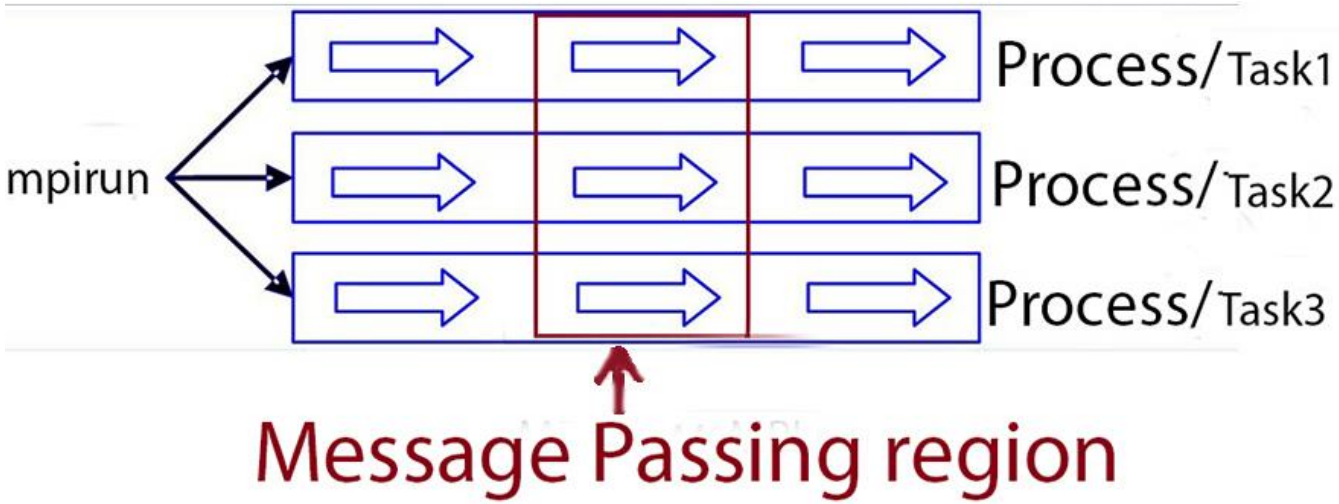
EJECUCION (para 4 procesos):

```
$ mpirun -np 4 python3 hola_mundo.py
Hola, mundo! Soy el proceso 1 de 4
Hola, mundo! Soy el proceso 2 de 4
Hola, mundo! Soy el proceso 3 de 4
Hola, mundo! Soy el proceso 0 de 4
```

- TRATE DE PENSAR EN PARALELO:
  - Al ejecutar **`$ mpirun -np N ...`** cada uno de los N procesos ejecuta una copia del código fuente
  - Si N es mayor que el número de procesos que permite un nodo, se obtiene un error. Para remediarlo, se puede "sobresuscribir" el nodo con más procesos. Esto se hace ejecutando mpirun así:
    `$ mpirun -np N –oversubscribe ...`
  - Cada variable se duplica N veces y puede tomar diferentes valores en los distintos procesos
  - Si se quiere chequear los valores de variables con print, hay que tener en cuenta:
    - No en todas las implementaciones de MPI se cumple que todos los nodos tengan acceso a E/S
    - En caso de que sí lo tengan, si chequea los valores de variables con print muestre siempre el rango del proceso en la salida de cada instrucción print

EJECUCIÓN Y PROGRAMACIÓN SPMD

- El comando de ejecución depende de la implementación concreta de MPI que se use, pero la esencia es la misma:
  - Se coloca una copia del ejecutable en cada procesador
  - Cada procesador comienza la ejecución de su copia del ejecutable
  - Procesos diferentes pueden ejecutar instrucciones distintas incluyendo en el programa una bifurcación condicional basada en el valor del rango del propio proceso.



MPI.Get_processor_name

name = MPI.Get_processor_name()

Devuelve el nombre del procesador desde donde la función fue llamada.

```
from mpi4py import MPI

# Establezco el comunicador global
comm = MPI.COMM_WORLD
# Qué numero de proceso soy?
rank = comm.Get_rank()
# Cuántos procesos hay en mi comunicador?
size = comm.Get_size()
# Desde donde me ejecuto?
nodo = MPI.Get_processor_name()

print (f"Hola, mundo! Soy el proceso {rank} de {size},
ejecutandome desde {nodo}")
```

FUNCIONES DE COMUNICACIÓN PUNTO A PUNTO

- Incluyen llamadas SEND y RECEIVE entre dos procesos.
- Existen dos categorías básicas de métodos send y receive:
  - Bloqueantes: Retornan cuando el send (o el receive) se ha completado:
    - send
    - recv
  - No bloqueantes: Retornan inmediatamente y queda al arbitrio del programador chequear o no si las llamadas se han completado:
    - isend (véase el anexo)
    - irecv
    - Para verificar si las llamadas no bloqueantes se han completado se emplean los tests (como métodos de un objeto request devuelto por isend/irecv):
      - test
      - wait (véase el anexo)
- Estudiaremos los send y receive bloqueantes estándar.

send


```
comm.send (dato, dest, tag)
```

- [IN **dato**]: el valor, cadena o lista de valores que queremos enviar
- [IN **dest**]: rango del proceso destino (entero)
- [IN **tag**]: etiqueta del mensaje (entero). Opcional en Python

Envía un mensaje conteniendo *dato* usando la etiqueta de mensaje *tag*, al proceso cuyo rango es *dest*, en el comunicador *comm*.

En la implementación de OpenMPI que usamos en el laboratorio, *send* no se bloquea si la cantidad de datos a enviar no rebasa los 4 KiB.

recv


```
dato = comm.recv(source, tag, status)
```

- [IN **source**]: rango del proceso fuente (entero) o MPI.ANY_SOURCE
- [IN **tag**]: etiqueta del mensaje (entero) o MPI.ANY_TAG .
  - Puede omitirse si se hizo así también en el emisor.
- [OUT **status**]: información de estado (estructura con tres campos que contienen el rango del proceso fuente, la etiqueta y el código de error producido)
  - Puede omitirse si no se piensa usar ninguno de los campos de status

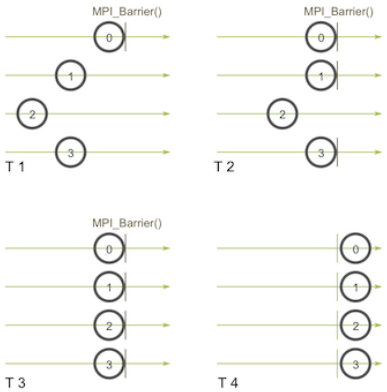
Bloquea un proceso hasta que recibe un mensaje del proceso cuyo rango es *source*, en el comunicador *comm*, con etiqueta de mensaje *tag*. El dato que contiene el mensaje es el resultado de la función.

Se pueden usar los comodines *MPI.ANY_SOURCE* y *MPI.ANY_TAG* para recibir mensajes. En ese caso, el *status* de salida puede ser usado para saber la fuente y etiqueta del mensaje recibido.

PROGRAMA saludos.py

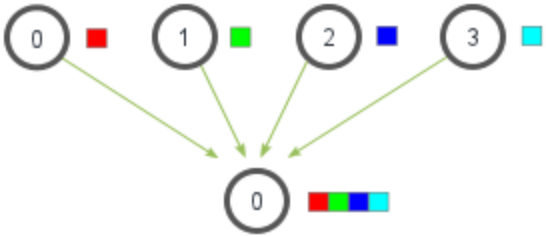

```python
from mpi4py import MPI

# Establezco el comunicador global
comm = MPI.COMM_WORLD

# Determino el rango de este proceso
mi_rango = comm.Get_rank()

# Determino el numero de procesos
p = comm.Get_size()

if mi_rango != 0:
    # Si no soy el proceso 0, creo el mensaje
    mensaje = f"Saludos desde el proceso {mi_rango}"
    # y se lo envio al proceso 0
    comm.send (mensaje, dest=0)
else:
    # si soy el proceso 0, itero por todos los números de procesos (fuente)
    for fuente in range(1,p):
        # Para cada fuente, recibo un mensaje desde él
        mensaje = comm.recv (source=fuente)
        # y lo imprimo
        print (mensaje)
```

Otras funciones de comunicación



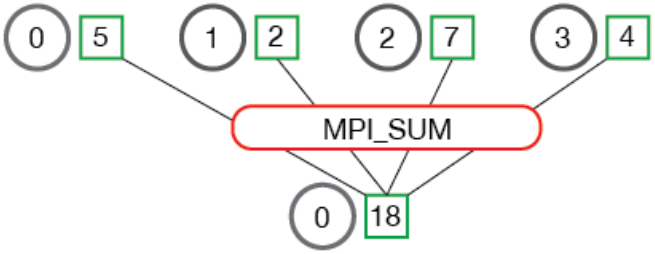
`comm.barrier()`

- Suspende la ejecución de cada proceso a medida que entran a ejecutar esta función. Sólo cuando todos los procesos del comunicador han entrado, se reanuda la ejecución de todos ellos. Permite sincronizar a todos los procesos.



`lista = comm.gather (dato, root)`

- Todos los procesos envían 'dato' al proceso cuyo rango es el especificado en root. Sólo el proceso de rango root puede iterar posteriormente la lista, que contiene todos los datos enviados.



`dato_reducido = comm.reduce (dato, root, op)`

- Todos los procesos envían 'dato' al proceso cuyo rango es el especificado en root. Con todos los datos enviados se realiza la operación especificada en op (MPI.SUM, MPI.PROD, MPI.MIN, MPI.MAX, etc). Sólo el proceso de rango root tiene acceso al valor de 'dato_reducido'.

Operaciones colectivas de comunicación

`comm.Scatter(datos, root)`

- Sirve para repartir datos desde un proceso raíz a todos los demás procesos. Divide una lista o vector en partes iguales según el número de procesos y envía una porción distinta a cada uno.

`comm.Bcast(datos, root)`

- El proceso raíz envía una copia completa e idéntica de la estructura de datos a todos los procesos del comunicador.

`comm.Gather(datos_locales, datos_finales, root)`

- Realiza la operación inversa a Scatter. Reúne los datos parciales de cada proceso y los pega en orden en una estructura final dentro del proceso raíz.

`comm.reduce(datos, op, root)`

- Recolecta datos de todos los procesos en un comunicador, los combina mediante una operación matemática o lógica (como MPI.SUM, MPI.MAX o MPI.MIN) y entrega el resultado final a un único proceso específico (típicamente el 0 o root).

`comm.Allreduce(datos, op):`

- Opera de manera similar a reduce, pero con la diferencia fundamental de que el resultado final de la reducción se envía a todos los procesos en lugar de solo al maestro.

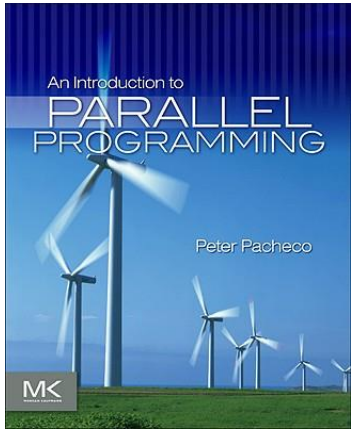
EVALUACIÓN DEL RENDIMIENTO

- MPI define una función para medir tiempos de ejecución con la que se puede evaluar el rendimiento de los programas
- Al ser una función del estándar MPI, tiene como ventaja su portabilidad
- **MPI.Wtime()**
  - Devuelve un número de segundos en punto flotante, que representan una referencia de tiempo (wall-clock time).
- Para medir la duración de un proceso, se puede llamar a *MPI.Wtime()* justo antes de que comience y justo despueś de que termine. A continuación, se calcula la diferencia entre los dos tiempos.
- Es aconsejable comenzar a medir el tiempo justo después de una barrera (llamar a comm.barrier() y terminar la medida después de la última llamada a un método de comunicaciones. Entonces, el proceso raíz procede a escribir el valor de la diferencia de ambos tiempos en pantalla)

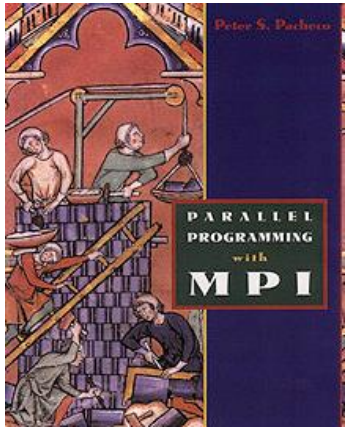
```
comm.barrier()
tinicial = MPI.Wtime()
# aquí se pone todo aquello que se quiere medir...
tfinal = MPI.Wtime()
if rank == 0:   # supondremos en este ejemplo que el proceso raíz es el 0
    print (f"Tiempo consumido: {tfinal-tinicial} seg.")
```

MPI: BIBLIOGRAFÍA

- "An Introduction to Parallel Programming", Peter S. Pacheco. Elsevier - Morgan Kaufmann, 2011.



- "Parallel Programming with MPI", Peter S. Pacheco. Elsevier - Morgan Kaufmann, 1996.



- "MPI: A Message-Passing Interface Standard". Version 2.2. Message Passing Interface Forum. September 4, 2009. http://www.mpi-forum.org/docs/mpi-2.2/mpi22-report.pdf

Anexo: conocimientos opcionales
Send no bloqueante


```
peticion = comm.isend (dato, dest, tag)
```

- Funciona como comm.send pero con estas consideraciones:
  - No se bloquea. El envío sigue sucediendo en segundo plano.
  - Devuelve un objeto de tipo Request (la variable peticion) que puede usarse más tarde para comprobar el estado del envío.
  - Mientras el envío está en curso, no debe alterarse la variable que contiene el dato

```
peticion.wait()
```

- Bloquea al proceso hasta que una operación isend anterior (la que devolvió el objeto peticion) realmente ha terminado.
  - Hasta que no termina la operación no se debe reusar o destruir la variable que contiene el dato que se envió.